

High Availability Web Architecture

AWS Cloud Security Project Report

Course Code: ONL5-ISS11-S5

Course Name: AWS Cloud Security

Instructor: Ahmed Attia

Team Members: Rana Abdelsalam, Heba Youssef, Menna Gamal

GitHub Repository

<https://github.com/Rana-Abdelsalamm/AWS-HA-Architecture.git>

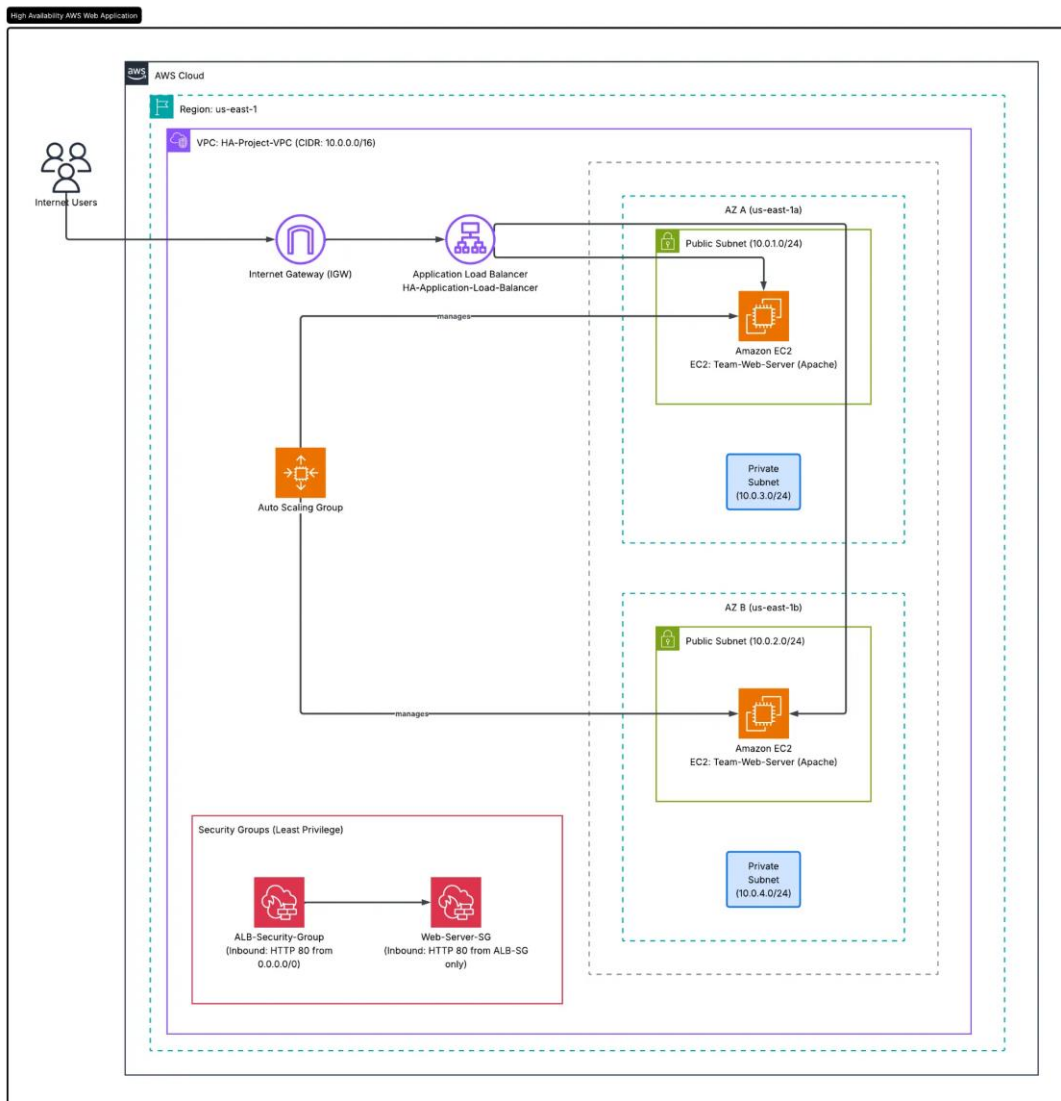
Table of Contents

Project Summary and Architecture Diagram	3
Implementation Evidence	4
1. CloudFormation Deployment Completion	4
2. VPC and Resource Map	5
3. EC2 Running Instances	6
4. Target Group Health	7
5. Active Application Load Balancer	8
6. End-to-End Live Web Page	9
CloudFormation Template (Infrastructure as Code)	10–15

Project Summary and Architecture Diagram

This project implements a highly available and fault-tolerant web architecture on Amazon Web Services using AWS CloudFormation as infrastructure as code. The solution provisions a custom Virtual Private Cloud with IPv4 CIDR block 10.0.0.0/16, public and private subnet tiers distributed across two Availability Zones in the us-east-1 Region, and the routing and security controls required for controlled web access.

Internet users reach an internet-facing Application Load Balancer through an Internet Gateway. The load balancer forwards HTTP traffic on port 80 to Apache web servers managed by an Auto Scaling Group. The group maintains a minimum of two instances and can scale to four, while the target group performs HTTP health checks on the root path. Together, multi-AZ placement, health-based routing, and automated instance replacement reduce the impact of an individual instance or Availability Zone failure and support continuous service availability.

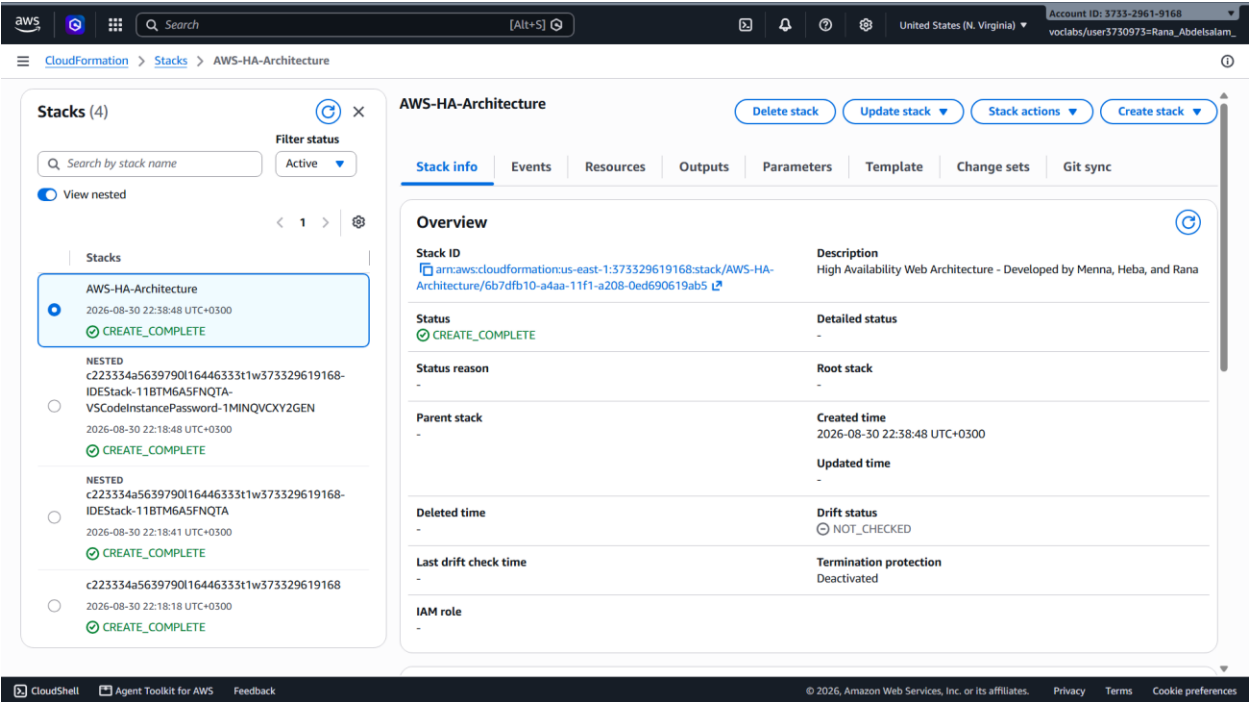


Logical architecture of the AWS high-availability web application.

Implementation Evidence

1. CloudFormation Deployment Completion

The CloudFormation console confirms that the AWS-HA-Architecture stack was created successfully with status `CREATE_COMPLETE`. This demonstrates that the infrastructure template was accepted, and its declared AWS resources were provisioned without a deployment failure.



CloudFormation stack in the `CREATE_COMPLETE` state.

2. VPC and Resource Map

The VPC console shows the deployed HA-Project-VPC as available with CIDR block 10.0.0.0/16, DNS resolution enabled, and DNS hostnames enabled. The resource map confirms four subnets across us-east-1a and us-east-1b, two route tables, and the HA-Project-IGW connection.

The screenshot displays the AWS VPC console for the VPC `vpc-0fa674f86ccdea1c` / HA-Project-VPC. The interface is divided into two main sections: Details and Resource map.

Details:

- VPC ID:** vpc-0fa674f86ccdea1c
- DNS resolution:** Enabled
- Main network ACL:** acl-Defa165d175297c9f
- IPv6 CIDR (Network border group):** -
- Encryption control ID:** -
- State:** Available
- Tenancy:** default
- Default VPC:** No
- Network Address Usage metrics:** Disabled
- Encryption control mode:** -
- Block Public Access:** Off
- DHCP option set:** dopt-0c1f6a11fa5a726f7
- IPv4 CIDR:** 10.0.0.0/16
- Route 53 Resolver DNS Firewall rule groups:** -
- DNS hostnames:** Enabled
- Main route table:** rtb-Odda08101d0734ed0
- IPv6 pool:** -
- Owner ID:** 373329619168

Resource map:

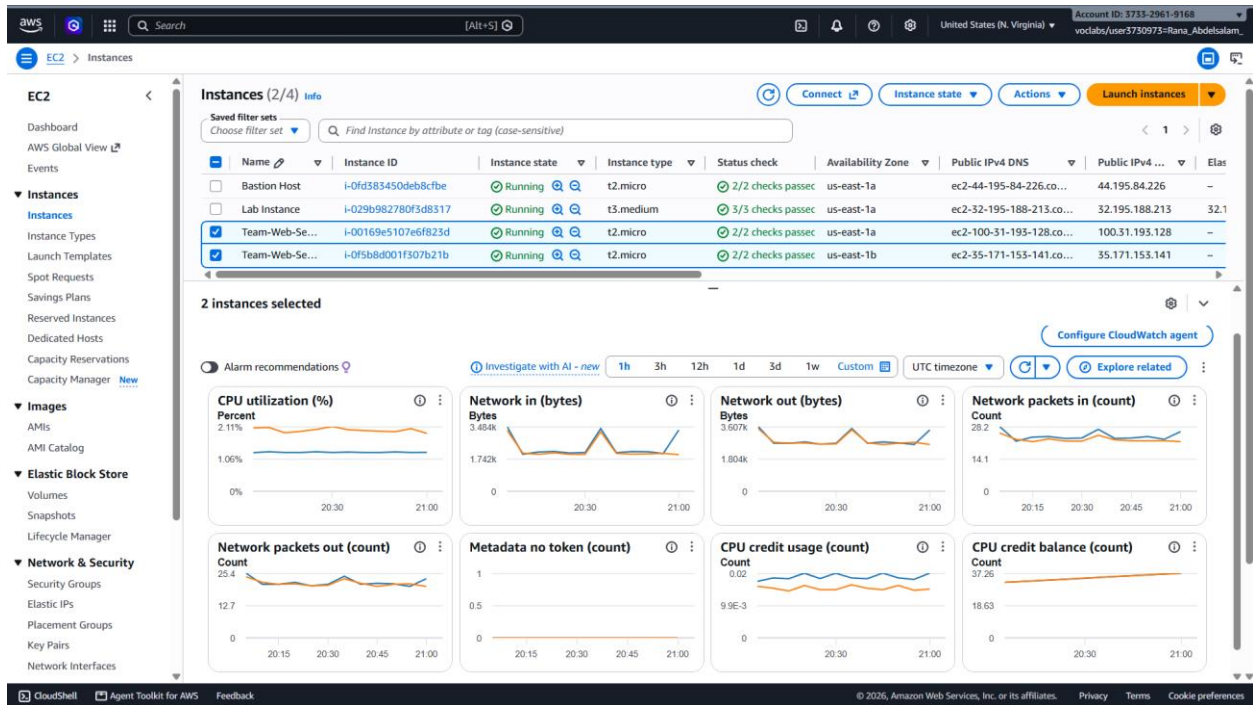
- VPC:** HA-Project-VPC
- Subnets (4):**
 - us-east-1a:** Public-Subnet-1, Private-Subnet-1
 - us-east-1b:** Public-Subnet-2, Private-Subnet-2
- Route tables (2):** Public-Route-Table, rtb-Odda08101d0734ed0
- Network Connections (1):** HA-Project-IGW

The resource map shows the VPC connected to four subnets across two Availability Zones (us-east-1a and us-east-1b). The subnets are connected to two route tables, which are then connected to the HA-Project-IGW (Internet Gateway).

Deployed VPC resources distributed across two Availability Zones.

3. EC2 Running Instances

The EC2 console presents two selected Team-Web-Server instances in the Running state. The instances use the t2.micro type, are placed in separate Availability Zones, and report passing status checks, providing runtime evidence of the Auto Scaling Group's baseline capacity.



Two running web-server instances across us-east-1a and us-east-1b.

4. Target Group Health

The target group is configured for HTTP traffic on port 80 and reports two total registered targets, with two healthy and zero unhealthy targets. The registered instances are distributed across us-east-1a and us-east-1b, confirming that the load-balancing tier has healthy back-end capacity in both zones.

The screenshot displays the AWS Management Console for the target group **AWS-HA-MyTar-V44OK9LUF21T**. The left sidebar shows the navigation menu with categories like Savings Plans, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area shows the details of the target group, including its ARN, target type (Instance), IP address type (IPv4), protocol (HTTP), and protocol version (HTTP1). It also shows the VPC and load balancer associated with the target group.

Details

arn:aws:elasticloadbalancing:us-east-1:373329619168:targetgroup/AWS-HA-MyTar-V44OK9LUF21T/78784b5caa924451

Target type: Instance

Protocol : Port: HTTP: 80

Protocol version: HTTP1

VPC: vpc-0fa674f86ccdea1c

IP address type: IPv4

Load balancer: AWS-HA-MyLoa-bjhPqKq9nbi

2 Total targets

2 Healthy

0 Unhealthy

0 Anomalous

0 Unused

0 Initial

0 Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (2) [Info](#)

Anomaly mitigation: Not applicable

Deregister | Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Admini...	Overri...	Launch...
☐	i-0f5b8d001f507b21b	Team-Web-Ser...	80	us-east-1b (us...	Healthy	-	No override.	No overri...	August 30...
☐	i-00169e5107e6f823d	Team-Web-Ser...	80	us-east-1a (us...	Healthy	-	No override.	No overri...	August 30...

Target group health status: two healthy targets and zero unhealthy targets.

5. Active Application Load Balancer

The Load Balancer console shows an internet-facing Application Load Balancer with status Active. Its resource map displays an HTTP:80 listener, forwarding rules to one target group, and two healthy targets across the two configured Availability Zones.

The screenshot displays the AWS Management Console for an Application Load Balancer named **AWS-HA-MyLoa-bjHPqKlq9nbi**. The console is in the **United States (N. Virginia)** region, and the user is **vocabio/user3730973-Rana_Aboelsalam_**.

Details:

- Load balancer type:** Application
- Status:** Active
- VPC:** vpc-0fa674f86ccdea1c
- Load balancer IP address type:** IPv4
- Scheme:** Internet-facing
- Hosted zone:** Z35SXDOTRQ7X7K
- Availability Zones:** subnet-0c8f4b9ac734d598a (us-east-1a (use1-az1)), subnet-05a55a3bccda128c6e (us-east-1b (use1-az2))
- Load balancer ARN:** arn:aws:elasticloadbalancing:us-east-1:373329619168:loadbalancer/app/AWS-HA-MyLoa-bjHPqKlq9nbi/4409c6993b3744ed
- DNS name:** AWS-HA-MyLoa-bjHPqKlq9nbi-732094206.us-east-1.elb.amazonaws.com (A Record)
- Date created:** August 30, 2026, 22:39 (UTC+03:00)

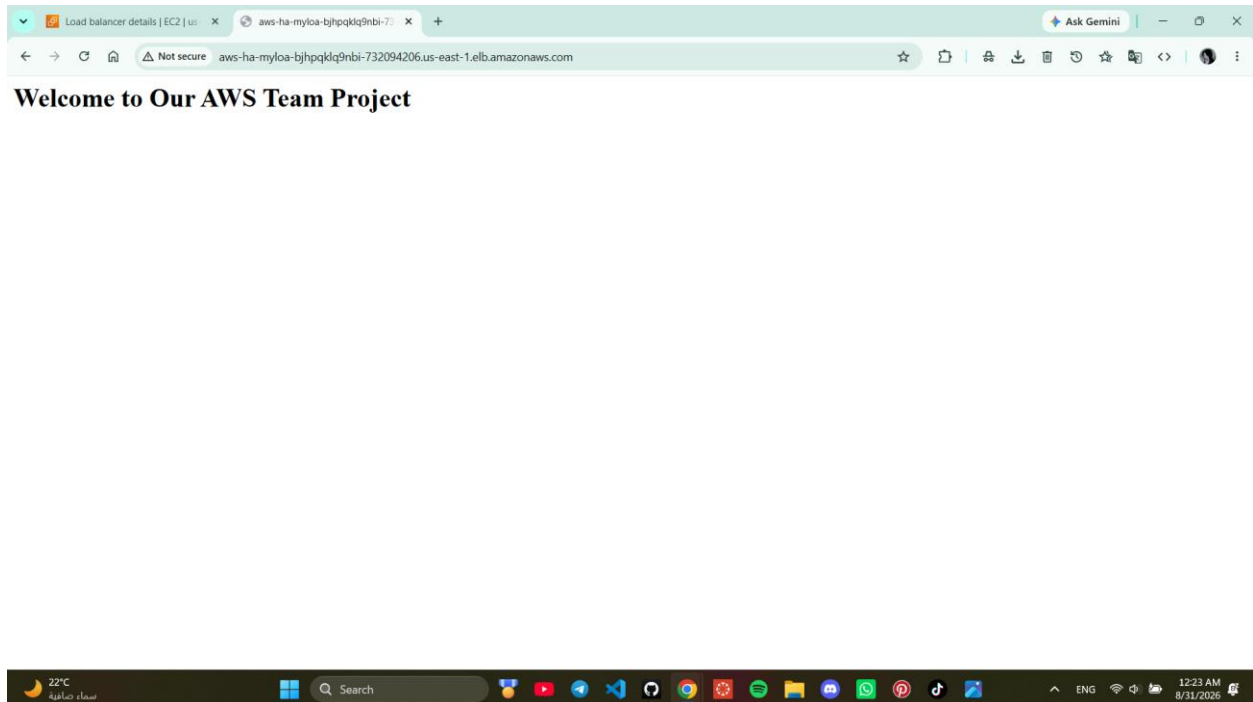
Resource map:

The resource map shows the architecture of the load balancer. It includes a **Listeners (1)** section with an **HTTP:80** listener. This listener is associated with **Rules (1)**, which has a **Priority default** and **Forward to target group**. The rule is associated with **Target groups (1)**, which has an **Instance, HTTP** target group named **AWS-HA-MyTar-V440K9LUF21T**. This target group is associated with **Targets (2)**, which includes two targets: **i-00169e5107e6f823d** (Port 80) and **i-0f5b8d001f307b21b** (Port 80). Both targets are marked as **Healthy**.

Active internet-facing ALB with HTTP listener and healthy targets.

6. End-to-End Live Web Page

The browser successfully reaches the Application Load Balancer DNS endpoint and renders the Apache-hosted page titled Welcome to Our AWS Team Project. This screenshot is the final end-to-end validation that client traffic can traverse the ALB and reach a healthy web-server target.



Live web page served through the Application Load Balancer endpoint.

CloudFormation Template (Infrastructure as Code)

The following appendix reproduces the complete YAML template used to define the architecture. It includes the custom VPC and subnet topology, Internet Gateway and routing, security groups, EC2 launch template, Application Load Balancer, listener, target group, and Auto Scaling Group.

Code:

AWSTemplateFormatVersion: '2010-09-09'

Description: 'High Availability Web Architecture - Developed by Menna, Heba, and Rana'

Resources:

```
# =====  
# 1. Menna's Section: VPC & Networking  
# =====
```

1. Main Virtual Private Cloud (VPC)

VPC:

Type: AWS::EC2::VPC

Properties:

CidrBlock: 10.0.0.0/16

EnableDnsSupport: true

EnableDnsHostnames: true

Tags:

- Key: Name

Value: HA-Project-VPC

2. Internet Gateway and Attachment

InternetGateway:

Type: AWS::EC2::InternetGateway

Properties:

Tags:

- Key: Name

Value: HA-Project-IGW

VPCGatewayAttachment:

Type: AWS::EC2::VPCGatewayAttachment

Properties:

VpcId: !Ref VPC

InternetGatewayId: !Ref InternetGateway

3. Public Subnets (For Load Balancer & Web Servers to access Internet)

PublicSubnet1:

Type: AWS::EC2::Subnet

Properties:

VpcId: !Ref VPC

CidrBlock: 10.0.1.0/24

AvailabilityZone: us-east-1a

MapPublicIpOnLaunch: true

Tags:

- Key: Name

Value: Public-Subnet-1

PublicSubnet2:

Type: AWS::EC2::Subnet

Properties:

VpcId: !Ref VPC

CidrBlock: 10.0.2.0/24

AvailabilityZone: us-east-1b

MapPublicIpOnLaunch: true

Tags:

- Key: Name

Value: Public-Subnet-2

4. Private Subnets (Currently not used for servers to allow Apache installation, but kept for architecture)

PrivateSubnet1:

Type: AWS::EC2::Subnet

Properties:

VpcId: !Ref VPC

CidrBlock: 10.0.3.0/24

AvailabilityZone: us-east-1a

MapPublicIpOnLaunch: false

Tags:

- Key: Name

Value: Private-Subnet-1

PrivateSubnet2:

Type: AWS::EC2::Subnet

Properties:

VpcId: !Ref VPC

CidrBlock: 10.0.4.0/24

AvailabilityZone: us-east-1b

MapPublicIpOnLaunch: false

Tags:

- Key: Name

Value: Private-Subnet-2

5. Public Route Table and Subnet Associations

PublicRouteTable:

Type: AWS::EC2::RouteTable

Properties:

VpcId: !Ref VPC

Tags:

- Key: Name

Value: Public-Route-Table

PublicRoute:

Type: AWS::EC2::Route

Properties:

RouteTableId: !Ref PublicRouteTable

DestinationCidrBlock: 0.0.0.0/0

GatewayId: !Ref InternetGateway

Subnet1Association:

Type: AWS::EC2::SubnetRouteTableAssociation

Properties:
 SubnetId: !Ref PublicSubnet1
 RouteTableId: !Ref PublicRouteTable

Subnet2Association:
 Type: AWS::EC2::SubnetRouteTableAssociation
 Properties:
 SubnetId: !Ref PublicSubnet2
 RouteTableId: !Ref PublicRouteTable

6. Security Groups

ALBSecurityGroup:
 Type: AWS::EC2::SecurityGroup
 Properties:
 GroupDescription: Allow HTTP traffic from everywhere to Load Balancer
 VpcId: !Ref VPC
 SecurityGroupIngress:
 - IpProtocol: tcp
 FromPort: 80
 ToPort: 80
 CidrIp: 0.0.0.0/0
 Tags:
 - Key: Name
 Value: ALB-Security-Group

InstanceSecurityGroup:
 Type: AWS::EC2::SecurityGroup
 Properties:
 GroupDescription: Allow HTTP traffic from ALB Security Group only
 VpcId: !Ref VPC
 SecurityGroupIngress:
 - IpProtocol: tcp
 FromPort: 80
 ToPort: 80
 SourceSecurityGroupId: !Ref ALBSecurityGroup
 Tags:
 - Key: Name
 Value: Web-Server-SG

=====
2. Heba's Section: EC2 Launch Template
=====

EC2 Launch Template for the Web Servers

MyLaunchTemplate:
 Type: AWS::EC2::LaunchTemplate
 Properties:
 LaunchTemplateName: WebServerTemplate
 LaunchTemplateData:

 # 1. Amazon Machine Image (Linux) and Instance Type
 ImageId: ami-0a3c3a20c09d6f377
 InstanceType: t2.micro

2. Security Group Attachment

SecurityGroupIds:

- !Ref InstanceSecurityGroup

3. User Data Script to automatically start Apache Web Server

UserData:

```
Fn::Base64: !Sub |
  #!/bin/bash
  yum update -y
  yum install -y httpd
  systemctl start httpd
  systemctl enable httpd
  echo "<h1>Welcome to Our AWS Team Project</h1>" >
```

/var/www/html/index.html

=====

3. Rana's Section: ALB & Auto Scaling

=====

1. Target Group to connect instances with the Load Balancer

MyTargetGroup:

Type: AWS::ElasticLoadBalancingV2::TargetGroup

Properties:

VpcId: !Ref VPC

Port: 80

Protocol: HTTP

TargetType: instance

HealthCheckIntervalSeconds: 30

HealthCheckPath: /

HealthCheckProtocol: HTTP

Tags:

- Key: Name

Value: HA-Target-Group

2. Application Load Balancer Configuration

MyLoadBalancer:

Type: AWS::ElasticLoadBalancingV2::LoadBalancer

Properties:

Subnets:

- !Ref PublicSubnet1

- !Ref PublicSubnet2

SecurityGroups:

- !Ref ALBSecurityGroup

Scheme: internet-facing

Tags:

- Key: Name

Value: HA-Application-Load-Balancer

3. ALB Listener for incoming traffic

MyListener:

Type: AWS::ElasticLoadBalancingV2::Listener

Properties:

DefaultActions:

- Type: forward

TargetGroupArn: !Ref MyTargetGroup

LoadBalancerArn: !Ref MyLoadBalancer

Port: 80

Protocol: HTTP

4. Auto Scaling Group Configuration

MyAutoScalingGroup:

Type: AWS::AutoScaling::AutoScalingGroup

Properties:

Adjusted to Public Subnets so servers can download Apache

VPCZoneIdentifier:

- !Ref PublicSubnet1

- !Ref PublicSubnet2

LaunchTemplate:

LaunchTemplateId: !Ref MyLaunchTemplate

Version: !GetAtt MyLaunchTemplate.LatestVersionNumber

MinSize: '2'

MaxSize: '4'

TargetGroupARNs:

- !Ref MyTargetGroup

Tags added so instances get a clear name when created

Tags:

- Key: Name

Value: Team-Web-Server

PropagateAtLaunch: true